Relatório Técnico — Pré-processador e Analisador Léxico µ-Assembly

Disciplina: Linguagens Formais, Autômatos e Compiladores
Universidade Católica de Brasília — 2º semestre de 2026
Aluno: Davi Castilho Lopess
Matrícula: UC22100801

Sumário

1. Estruturas de dados
2. Funções implementadas
3. Autômato Finito Determinístico (AFD)
4. Estratégia de controle de linha e coluna
5. Tratamento e recuperação de erros (catálogo de erros léxicos)
6. Tabela de Símbolos
7. Testes realizados
8. Saídas geradas pelo analisador
9. Conteúdo final da Tabela de Símbolos
10. Decisões de projeto

- Anexo A — Tabela de Símbolos completa do teste T01

Compilação e execução

De acordo com a pastar uassembly.zip que foi anexada na atividade, a compilação e execução segue dessa forma:

```
gcc main.c preprocessador.c lexico.c -o main.exe
./main.exe .<entrada.asm> <saida.pre> <saida.lex>
```

Exemplo: `./main.exe teste.asm teste.pre teste.lex` gera `teste.pre` (código pré-processado), `teste.lex` (tokens), `teste.ts` (Tabela de Símbolos) e `teste.err` (erros léxicos).

Se a quantidade de argumentos for diferente de três, ou se algum arquivo não puder ser aberto ou criado, o programa exibe uma mensagem de erro em `stderr` e encerra com código de saída 1, sem prosseguir para as etapas seguintes.

---

1. Estruturas de dados

1.1 `Token` (token.h)

Representa uma unidade léxica reconhecida, exatamente com os campos exigidos pelo documento:

```
typedef struct {
    char nome[50];     /* categoria do token, ex.: "INS_LI", "REG" */
    char lexema[100];  /* texto reconhecido, ex.: "$t0"            */
    int linha;         /* linha onde o lexema começa               */
    int coluna;        /* coluna onde o lexema começa              */
} Token;
```

A mesma estrutura é reaproveitada para representar um erro léxico: o campo `nome` guarda o tipo do erro (ex.: `ERRO_REGISTRADOR_INVALIDO`) e `lexema` guarda o trecho relacionado. Isso é possível porque tokens e erros têm o mesmo formato de saída (`<nome, lexema> linha coluna`).

1.2 `Simbolo` (token.h)

Uma entrada da Tabela de Símbolos:

```
typedef struct {
    char lexema[100];
    char categoria[30];   /* "diretiva", "instrução", "registrador"
                             ou "identificador/rótulo"             */
    int primeiraLinha;    /* 0 até a primeira ocorrência real       */
    int primeiraColuna;
} Simbolo;
```

1.3 `TabelaSimbolos` e `ListaErros` (lexico.c)

Ambas são arrays dinâmicos com a mesma organização:

```
typedef struct { Simbolo *itens; int quantidade; int capacidade; } TabelaSimbolos;
typedef struct { Token   *itens; int quantidade; int capacidade; } ListaErros;
```

`quantidade` é o número de elementos preenchidos e `capacidade` é o espaço alocado. Quando `quantidade` alcança `capacidade`, a capacidade é dobrada com `realloc`. Assim, não há limite fixo de símbolos ou de erros. Ambas ficam em variáveis `static` de `lexico.c`: apenas esse módulo as manipula diretamente, e `main.c` acessa seu conteúdo somente por meio de `GravarTabelaSimbolos` e `GravarErros`.

1.4 Tabelas de palavras reservadas (lexico.c)

Três vetores constantes, copiados do documento:

- `DIRETIVAS[]`: pares `{lexema, token}` com as 10 diretivas (ex.: `{".data", "DIR_DATA"}`);
- `INSTRUCOES[]`: pares `{lexema, token}` com as 36 instruções e pseudoinstruções (ex.: `{"add", "INS_ADD"}`);
- `REGISTRADORES_NOMEADOS[]`: os 31 nomes de registradores (`$zero`, `$v0`, ..., `$ra`). As formas numéricas `$0` a `$31` são geradas por um laço na inicialização.

A quantidade de elementos de cada vetor é calculada com `sizeof(vetor) / sizeof(vetor[0])`, de modo que acrescentar uma entrada não exige atualizar nenhum contador manualmente.

1.5 Estado de leitura (lexico.c)

Variável	Significado
`linhaAtual`	linha do próximo caractere a ser lido
`colunaAtual`	coluna do próximo caractere a ser lido
`colunaAnterior`	coluna antes da última leitura (permite desfazê-la)

1.6 Pré-processador (preprocessador.c)

O pré-processador trabalha linha a linha com dois buffers de `TAM_MAX_LINHA` (4096) caracteres: um para a linha lida e outro para a linha processada. Uma variável de estado `dentroString` indica se o caractere atual está dentro de uma cadeia.

---

2. Funções implementadas

2.1 main.c

Função	Descrição
`main`	Valida os 3 argumentos, deriva os nomes de `.ts` e `.err`, abre os arquivos na ordem correta e chama `PreProcessamento`, `AnaliseLexica`, `GravarTabelaSimbolos` e `GravarErros`. Trata falhas de abertura com mensagem clara e encerramento controlado.
`trocarExtensao`	Copia um nome de arquivo substituindo a extensão após o último `.` (ex.: `teste.lex` → `teste.ts`).

O arquivo `.pre` é escrito, fechado e **reaberto em modo leitura** antes da análise léxica, em vez de reaproveitar o mesmo `FILE*`, evitando problemas de alternância entre escrita e leitura.

2.2 preprocessador.c

Função	Descrição
`PreProcessamento` (pública)	Lê cada linha, processa e grava apenas as linhas não vazias, sempre terminadas por `\n`.
`lerLinha`	Lê uma linha caractere a caractere, tratando `\n`, `\r\n` e `\r` de forma uniforme.
`processarLinha`	Remove comentários e normaliza espaços e tabulações fora de strings; copia strings sem alteração, respeitando `\"` escapado.

Os arquivos são abertos em modo binário (`"rb"`/`"wb"`) para que o tratamento das quebras de linha seja feito pelo próprio programa, igual em qualquer sistema operacional.

2.3 lexico.c — funções públicas (declaradas em lexico.h)

Função	Descrição
`AnaliseLexica(FILE *in, FILE *out)`	Reinicia a Tabela de Símbolos e a lista de erros, pré-carrega as palavras reservadas e executa o laço principal do AFD, gravando os tokens em `out` e terminando com `TK_EOF`.
`GravarTabelaSimbolos(FILE *out)`	Grava a Tabela de Símbolos em CSV (`LEXEMA,CATEGORIA,LINHA,COLUNA`).
`GravarErros(FILE *out)`	Grava um erro por linha (`<tipo, lexema> linha coluna`) ou a mensagem `Nenhum erro lexico encontrado.`

2.4 lexico.c — funções internas (`static`)

Tabela de Símbolos e erros

Função	Descrição
`inicializarTabela` / `liberarTabela`	Alocam e liberam a Tabela de Símbolos.
`buscarSimbolo`	Busca linear por lexema; devolve ponteiro para a entrada ou `NULL`.
`inserirOuAtualizarSimbolo`	Insere um símbolo novo ou, se já existir com posição `0,0`, registra a primeira ocorrência real.
`inicializarErros` / `liberarErros`	Alocam e liberam a lista de erros.
`registrarErro`	Acrescenta um erro à lista (sem verificação de duplicidade).

Função	Descrição
`buscarTokenReservado`	Procura um lexema em `DIRETIVAS` ou `INSTRUCOES` e devolve o nome do token.
`popularPalavrasReservadas`	Insere as 109 palavras reservadas na tabela com posição `0,0`.

Leitura e classificação de caracteres

Função	Descrição
`proximoCaractere`	Única função que consome caracteres do arquivo; informa a posição do caractere lido e atualiza linha e coluna.
`devolverCaractere`	Desfaz a última leitura (`ungetc`), restaurando linha e coluna.
`espiarCaractere`	Consulta o próximo caractere sem consumi-lo e sem alterar a posição.
`ehLetra`, `ehDigito`, `ehDigitoHex`	Classificação de caracteres.
`ehDelimitador`	Indica se um caractere pode seguir um identificador.
`paraMinusculas`	Normalização para minúsculas.
`escreverToken`	Grava um token no formato `<nome, lexema> linha coluna`.

Estados do AFD

Função	Estados que implementa
`consumirComentario`	`q_com`
`reconhecerIdentificadorOuInstrucao`	`q_id`
`reconhecerDiretiva`	`q_dir0`, `q_dir`
`reconhecerRegistrador`	`q_reg0`, `q_reg`
`reconhecerNumero`	`q_neg`, `q_zero`, `q_dec`, `q_hex0`, `q_hex`
`reconhecerString`	`q_str`, `q_esc`
`consumirRestoDaPalavra`, `acrescentarAoLexema`	auxiliares de recuperação e montagem do lexema

---

3. Autômato Finito Determinístico (AFD)

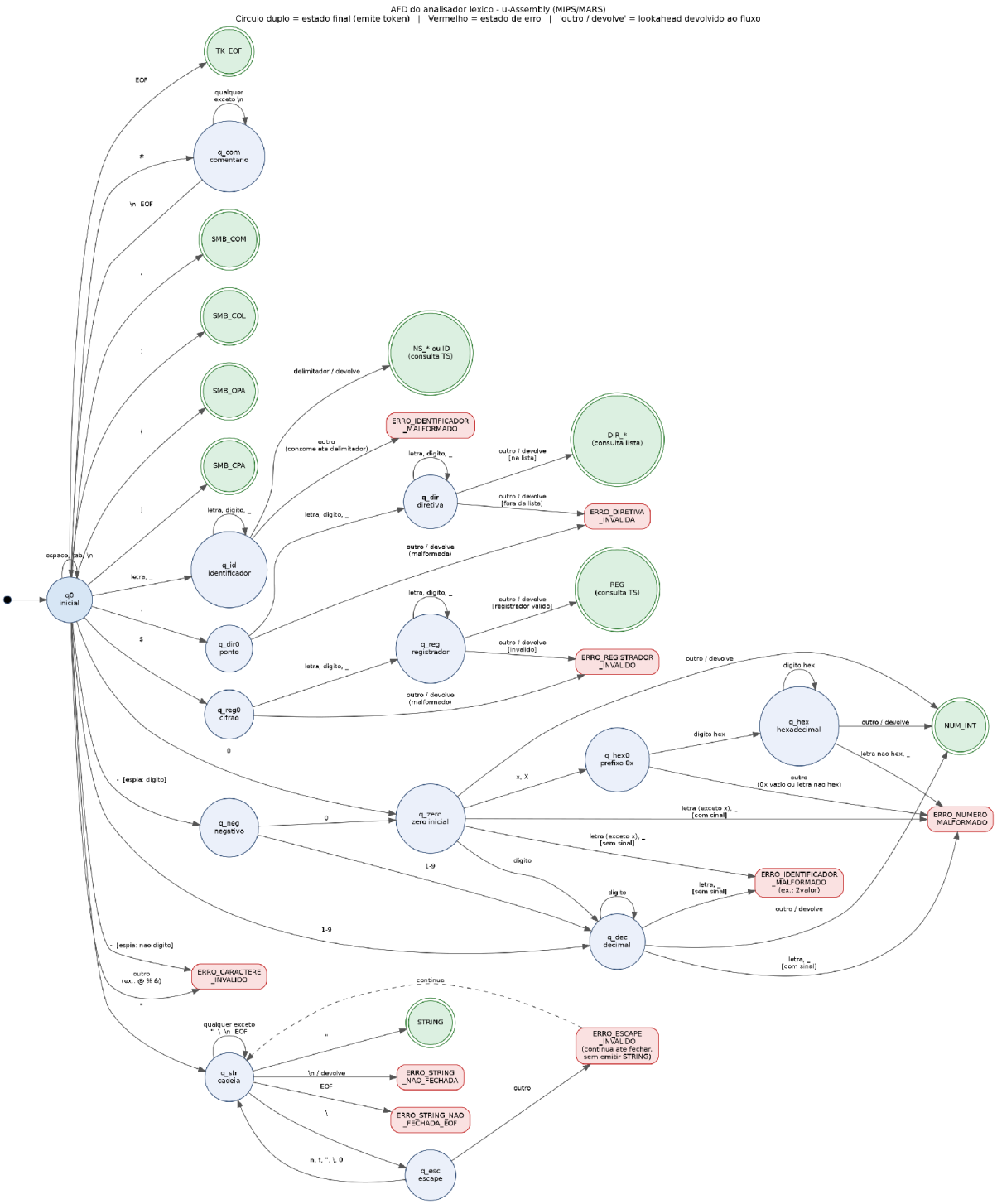


Figura 1 — AFD do analisador léxico

O diagrama foi descrito em Graphviz (`docs/afd.dot`) e renderizado com `dot -Tpng afd.dot -o afd.png`. Também há uma versão vetorial em `docs/afd.svg`.

3.1 Funcionamento geral

O laço principal de `AnaliseLexica` corresponde ao **estado inicial `q0`**. A cada iteração ele lê um caractere e, com base apenas nesse primeiro caractere, decide para qual estado seguir:

Primeiro caractere	Estado	Resultado
espaço, tabulação, \n	permanece em q0	nenhum token
#	q_com	descarta até o fim da linha
, : ( )	estado final de símbolo	SMB_COM, SMB_COL, SMB_OPA, SMB_CPA
letra ou _	q_id	INS_*, ID ou erro
.	q_dir0	DIR_* ou erro
$	q_reg0	REG ou erro
dígito	q_zero ou q_dec	NUM_INT ou erro
- seguido de dígito	q_neg	NUM_INT ou erro
"	q_str	STRING ou erro
EOF	estado final	TK_EOF
qualquer outro	estado de erro	ERRO_CARACTERE_INVALIDO

Cada estado de lexema de tamanho variável segue o mesmo procedimento: (1) coloca no buffer o caractere que disparou o estado; (2) consome os caracteres que pertencem ao lexema; (3) devolve ao fluxo o primeiro caractere que não pertence, pois ele inicia o próximo lexema; (4) classifica o lexema completo.

3.2 Lookahead

Duas transições dependem de olhar um caractere adiante sem consumi-lo (`espiarCaractere`): o `-`, que só inicia um número se o próximo caractere for um dígito, e o `0`, que só inicia um hexadecimal se o próximo for `x` ou `X`. Nas transições marcadas como "outro / devolve", o caractere que encerrou o lexema é devolvido com `devolverCaractere`.

3.3 Consulta às tabelas

Conforme as observações da seção 3 do documento, a classificação final consulta as palavras reservadas:

- lexema formado por letras: comparado com `INSTRUCOES` (em minúsculas). Se estiver na lista, gera o token específico (ex.: `INS_ADD`); caso contrário, gera `ID`;
- lexema iniciado por `.`: comparado com `DIRETIVAS`;
- lexema iniciado por `$`: procurado na própria Tabela de Símbolos, que contém desde a inicialização todos os registradores válidos com categoria `registrador`.

3.4 Estados e transições

Estado	Descrição	Transições
q_com	comentário	qualquer caractere exceto \n → q_com; \n ou EOF → q0
q_id	identificador	letra, dígito, _ → q_id; delimitador → final (INS_* ou ID); outro → erro de identificador malformado
q_dir0	ponto lido	letra, dígito, _ → q_dir; outro → erro (diretiva malformada)
q_dir	diretiva	letra, dígito, _ → q_dir; outro → final DIR_* se estiver na lista, senão erro
q_reg0	cifrão lido	letra, dígito, _ → q_reg; outro → erro (registrador malformado)
q_reg	registrador	letra, dígito, _ → q_reg; outro → final REG se válido, senão erro
q_neg	sinal negativo	0 → q_zero; 1-9 → q_dec

Estado	Descrição	Transições
`q_zero`	zero inicial	x/X → `q_hex0`; dígito → `q_dec`; letra → erro; outro → final `NUM_INT`
`q_dec`	decimal	dígito → `q_dec`; letra ou _ → erro; outro → final `NUM_INT`
`q_hex0`	prefixo `0x`	dígito hexadecimal → `q_hex`; outro → erro (`0x` vazio ou letra não hexadecimal)
`q_hex`	hexadecimal	dígito hexadecimal → `q_hex`; letra não hexadecimal ou _ → erro; outro → final `NUM_INT`
`q_str`	cadeia	" → final `STRING`; \ → `q_esc`; \n → erro; EOF → erro; outro → `q_str`
`q_esc`	sequência de escape	n, t, ", \, 0 → `q_str`; outro → erro de escape (continua em `q_str`)

---

4. Estratégia de controle de linha e coluna

1. **Uma única porta de leitura.** Todo caractere é lido por `proximoCaractere`. Nenhuma outra função chama `fgetc` para consumir caracteres, o que garante que a posição nunca fique desatualizada.
2. **Posição do caractere lido.** Antes de avançar, `proximoCaractere` informa ao chamador a linha e a coluna **onde o caractere estava**. O estado que inicia um lexema guarda essa posição, que é a posição inicial do token. Não há ajustes do tipo "coluna − 1" espalhados pelo código.
3. **Avanço.** Ao ler `\n`, a linha é incrementada e a coluna volta para 1. Qualquer outro caractere incrementa a coluna em 1. Espaços e tabulações não geram token, mas atualizam a posição.
4. **Desfazer uma leitura.** Antes de cada avanço, a coluna atual é guardada em `colunaAnterior`. `devolverCaractere` restaura essa coluna e, se o caractere devolvido for `\n`, também decrementa a linha. `espiarCaractere` não altera a posição.
5. **Fim de arquivo.** Ao encontrar EOF, é emitido `TK_EOF` com a posição em que o fim foi encontrado. Como o `.pre` sempre termina com `\n`, essa posição é a coluna 1 da linha seguinte à última linha do programa, como no exemplo da seção 4.1 do documento.
6. **Referência.** As posições se referem ao arquivo pré-processado (ver seção 10.2).

Exemplo (linha 6 do `.pre` do teste T01, `li $t0, 5`): `li` na coluna 1, `$t0` na coluna 4 (após `l`, `i` e o espaço), `,` na coluna 7 e `5` na coluna 9.

---

5. Tratamento e recuperação de erros

5.1 Erros de execução (main.c)

Situação	Comportamento
Quantidade de argumentos diferente de 3	Mostra a forma de uso em `stderr` e encerra com código 1.
Arquivo de entrada inexistente ou sem permissão	Mensagem indicando o arquivo e encerramento com código 1.
Falha ao criar `.pre`, `.lex`, `.ts` ou `.err`	Mensagem indicando o arquivo; arquivos já abertos são fechados antes do encerramento.

5.2 Catálogo de erros léxicos

Todos os erros são acumulados na lista de erros e gravados no `.err` ao final da análise, no formato `<tipo_erro, lexema> linha coluna`. Linha e coluna indicam o início do lexema com erro (no escape inválido, a posição da barra). As saídas abaixo foram obtidas dos testes T07, T08 e T09 (seção 7).

Tipo	Quando ocorre	Exemplo	Saída real no `.err`
`ERRO_CARACTERE_INVALIDO`	caractere que não inicia nenhum lexema da linguagem	`add $t1, $t0, @`	`<ERRO_CARACTERE_INVALIDO, @> 4 15`
		`li $t3, % 2`	`<ERRO_CARACTERE_INVALIDO, %> 8 9`
	`-` não seguido imediatamente de dígito	`sub $t4, $t0, - 5`	`<ERRO_CARACTERE_INVALIDO, -> 9 15`
`ERRO_IDENTIFICADOR_MALFORMADO`	identificador colado a caractere não delimitador	`soma-total: .word 1`	`<ERRO_IDENTIFICADOR_MALFORMADO, soma-total> 4 1`
		`inicio@: .word 2`	`<ERRO_IDENTIFICADOR_MALFORMADO, inicio@> 5 1`
	dígito seguido de letras	`2valor: .word 3`	`<ERRO_IDENTIFICADOR_MALFORMADO, 2valor> 6 1`
`ERRO_NUMERO_MALFORMADO`	letra não hexadecimal após `0x`	`li $t0, 0xG1`	`<ERRO_NUMERO_MALFORMADO, 0xG1> 7 9`
	prefixo `0x` sem dígitos	`li $t1, 0x`	`<ERRO_NUMERO_MALFORMADO, 0x> 8 9`
	número negativo seguido de letras	`li $t2, -2abc`	`<ERRO_NUMERO_MALFORMADO, -2abc> 9 9`
`ERRO_DIRETIVA_INVALIDA`	lexema com `.` fora do subconjunto	`.dados`	`<ERRO_DIRETIVA_INVALIDA, .dados> 1 1`
	ponto isolado (diretiva malformada)	`. word 5`	`<ERRO_DIRETIVA_INVALIDA, .> 2 1`
`ERRO_REGISTRADOR_INVALIDO`	nome fora do conjunto admitido	`li $t10, 5`	`<ERRO_REGISTRADOR_INVALIDO, $t10> 5 4`
	número fora de `$0`–`$31`	`move $32, $x1`	`<ERRO_REGISTRADOR_INVALIDO, $32> 6 6`
	nome inexistente	`move $32, $x1`	`<ERRO_REGISTRADOR_INVALIDO, $x1> 6 11`
	cifrão isolado (registrador malformado)	`add $t2, $, $t0`	`<ERRO_REGISTRADOR_INVALIDO, $> 7 10`
`ERRO_STRING_NAO_FECHADA`	quebra de linha antes da aspa de fechamento	`.asciiz "Mensagem nao encerrada`	`<ERRO_STRING_NAO_FECHADA, "Mensagem nao encerrada> 2 13`
	barra no fim da linha	`.asciiz "Termina com barra\`	`<ERRO_STRING_NAO_FECHADA, "Termina com barra\> 4 13`
`ERRO_STRING_NAO_FECHADA_EOF`	fim do arquivo antes da aspa de fechamento	`.asciiz "sem fim` (sem \n final)	`<ERRO_STRING_NAO_FECHADA_EOF, "sem fim> 2 14`

Tipo	Quando ocorre	Exemplo	Saída real no `.err`
`ERRO_ESCAPE_INVALIDO`	barra seguida de caractere diferente de `n`, `t`, `"`, `\`, `0`	`.asciiz "Escape \q invalido"`	`<ERRO_ESCAPE_INVALIDO, \q> 3 21`

O erro de fim de arquivo não pode ser produzido pelo `main.exe`, pois o pré-processador sempre termina as linhas com `\n`. Ele foi testado chamando `AnaliseLexica` diretamente sobre um arquivo sem quebra de linha final (`.data` / `msg: .asciiz "sem fim"`).

5.3 Estratégia de recuperação

O analisador segue a estratégia de **modo pânico no nível do lexema**: ao detectar um erro, descarta apenas o lexema defeituoso e retoma a análise a partir do caractere seguinte. Todos os erros do arquivo são registrados, e não apenas o primeiro, como exige o documento.

Erro	Recuperação
Caractere inválido	descarta apenas o caractere
Identificador malformado	consome até o próximo delimitador (espaço, `,`, `:`, `(`, `)`, `#`, `"`, `\n` ou EOF)
Número malformado	consome o restante da palavra (letras, dígitos, _)
Diretiva ou registrador inválido	o lexema inteiro já foi consumido pelo estado
Escape inválido	continua lendo a cadeia até a aspa de fechamento; a cadeia não gera `STRING`
Cadeia não fechada	devolve o `\n` ao fluxo e retoma na linha seguinte

Com isso, um único defeito gera um único registro. Por exemplo, `0xG1` produz um erro, e não um erro para `0x` seguido de um `ID G1`; e `.dados` produz um erro, e não um ponto inválido seguido de um `ID dados`. Nos testes T07 a T09, os demais tokens de cada linha com erro continuaram sendo reconhecidos normalmente.

Lexemas com erro não geram token no `.lex` e não são inseridos na Tabela de Símbolos.

---

6. Tabela de Símbolos

1. **Inicialização.** A cada execução de `AnaliseLexica`, a tabela é recriada e recebe as 10 diretivas, as 36 instruções, os 31 registradores nomeados e as 32 formas numéricas (`$0` a `$31`), totalizando 109 símbolos, todos com posição `0,0`.
2. **Primeira ocorrência.** Quando uma palavra reservada aparece no programa, sua posição `0,0` é substituída pela posição da primeira ocorrência. Ocorrências posteriores não alteram a entrada.
3. **Identificadores.** Cada identificador válido é inserido com categoria `identificador/rótulo` na primeira vez em que aparece, seja em uma declaração (`laco:`) ou em uma referência (`j laco`).
4. **Unicidade.** Antes de inserir, `buscarSimbolo` verifica se o lexema já existe; um mesmo lexema nunca aparece duas vezes.
5. **Normalização.** Instruções, diretivas e registradores são gravados em minúsculas (`ADD`, `.DATA` e `$T1` ocupam as entradas `add`, `.data` e `$t1`). Identificadores são sensíveis a maiúsculas: `inicio` e `Inicio` são entradas distintas.
6. **O que não entra.** Números, cadeias de caracteres, símbolos de pontuação e lexemas com erro.

7. **Saída.** O arquivo `.ts` é gravado em CSV com o cabeçalho `LEXEMA,CATEGORIA,LINHA,COLUNA`, na ordem de inserção: primeiro as palavras reservadas e, ao final, os identificadores do programa.

---

7. Testes realizados

Os arquivos de entrada estão em `testes/entradas/` e as saídas (`.pre`, `.lex`, `.ts`, `.err`) em `testes/saidas/`. Cada teste foi executado com:

```
./main.exe testes/entradas/<teste>.asm testes/saidas/<teste>.pre testes/saidas/<teste>.lex
```

7.1 Casos de teste exigidos pela seção 5 do documento

Teste	Exigência atendida	Conteúdo	Tokens	Erros
T01_correto_soma	programa correto 1	soma, `la`, `move`, `syscall`, `.globl`	48	0
T02_correto_laco	programa correto 2	laço, `j`, `jal`, `jr`, `blt`, `bne`, `mul`, `slt`, `nop`, referência antecipada	55	0
T03_correto_memoria	programa correto 3; acesso à memória	`lw`, `sw`, `lb`, `sb`, `lh`, `sh` com `0($t1)`, `4($t1)`, `-8($fp)`, `($t1)`	78	0
T04_comentarios	teste com comentários	comentários em linha inteira, após diretiva, rótulo, instrução e string; # dentro de string; comentário colado	14	0
T05_strings	teste com cadeias	as 5 sequências de escape, espaços internos, acentos, cadeia vazia, `.ascii` e `.asciiz`	30	0
T06_numeros	teste com decimais e hexadecimais	decimais, negativos, hexadecimais (`0x`, `0X`, maiúsculas e minúsculas) e hexadecimais negativos	57	0
T07_erros_caracteres_registradores	programa com erros 1	`@`, `%`, `-` isolado, `$t10`, `$32`, `$x1`, `$`	33	7
T08_erros_strings_numeros	programa com erros 2	cadeias não fechadas, escape inválido, `0xG1`, `0x`, `-2abc`	27	6
T09_erros_diretivas_identificadores	programa com erros 3	`.dados`, `.` isolado, `soma-total`, `inicio@`, `2valor`	24	5

A contagem de tokens inclui `TK_EOF`. Todos os erros registrados nos testes T07 a T09 estão listados no catálogo da seção 5.2.

7.2 Exemplos fornecidos pelo professor

Os 20 arquivos `Exemplo01.asm` a `Exemplo20.asm` também foram executados. Dezenove não apresentam erros léxicos. O `Exemplo05.asm` gera `<ERRO_REGISTRADOR_INVALIDO, $t99> 4 15`, que é o resultado esperado: o próprio arquivo indica que `$t99` não é um registrador válido (o pré-processador não deve acusá-lo, mas o analisador léxico deve).

7.3 Testes adicionais durante o desenvolvimento

Durante a implementação de cada estado, foram executados testes específicos, entre eles: diretivas em maiúsculas (`.DATA`, `.Text`); instrução em maiúsculas (`ADD`); rótulos `inicio` e `Inicio`; `add:` e `syscall:` (sem erro léxico, ver seção 10.4); `$T1`, `$01` e `$zero`; `0XaB`; o limite de coluna em linhas com vários erros; e a regressão completa dos 20 exemplos após cada alteração.

---

8. Saídas geradas pelo analisador

8.1 Teste T01 — arquivo `.pre`

```
.data
msg: .asciiz "Resultado: "
.text
.globl main
main:
li $t0, 5
li $t1, 10
add $t2, $t0, $t1
la $a0, msg
li $v0, 4
syscall
move $a0, $t2
li $v0, 1
syscall
li $v0, 10
syscall
```

8.2 Teste T01 — arquivo `.lex`

```
<DIR_DATA, .data> 1 1
<ID, msg> 2 1
<SMB_COL, :> 2 4
<DIR_ASCIIZ, .asciiz> 2 6
<STRING, "Resultado: "> 2 14
<DIR_TEXT, .text> 3 1
<DIR_GLOBL, .globl> 4 1
<ID, main> 4 8
<ID, main> 5 1
<SMB_COL, :> 5 5
<INS_LI, li> 6 1
<REG, $t0> 6 4
<SMB_COM, ,> 6 7
<NUM_INT, 5> 6 9
<INS_LI, li> 7 1
<REG, $t1> 7 4
<SMB_COM, ,> 7 7
<NUM_INT, 10> 7 9
<INS_ADD, add> 8 1
<REG, $t2> 8 5
<SMB_COM, ,> 8 8
<REG, $t0> 8 10
<SMB_COM, ,> 8 13
<REG, $t1> 8 15
<INS_LA, la> 9 1
<REG, $a0> 9 4
<SMB_COM, ,> 9 7
<ID, msg> 9 9
<INS_LI, li> 10 1
<REG, $v0> 10 4
<SMB_COM, ,> 10 7
<NUM_INT, 4> 10 9
<INS_SYSCALL, syscall> 11 1
<INS_MOVE, move> 12 1
<REG, $a0> 12 6
<SMB_COM, ,> 12 9
```

```
<REG, $t2> 12 11
<INS_LI, li> 13 1
<REG, $v0> 13 4
<SMB_COM, ,> 13 7
<NUM_INT, 1> 13 9
<INS_SYSCALL, syscall> 14 1
<INS_LI, li> 15 1
<REG, $v0> 15 4
<SMB_COM, ,> 15 7
<NUM_INT, 10> 15 9
<INS_SYSCALL, syscall> 16 1
<TK_EOF, EOF> 17 1
```

8.3 Teste T01 — arquivo `.err`

```
Nenhum erro lexico encontrado.
```

8.4 Teste T03 — acesso à memória (trecho)

Para `lw $t0, 0($t1)` (linha 7 do `.pre`):

```
<INS_LW, lw> 7 1
<REG, $t0> 7 4
<SMB_COM, ,> 7 7
<NUM_INT, 0> 7 9
<SMB_OPA, (> 7 10
<REG, $t1> 7 11
<SMB_CPA, )> 7 14
```

8.5 Teste T05 — cadeias de caracteres

```
<STRING, "Texto simples"> 2 13
<STRING, "Espacos    internos   preservados"> 3 13
<STRING, "Quebra\nTabulacao\t"> 4 13
<STRING, "Aspas \"internas\" e barra \\"> 5 13
<STRING, "Nulo explicito\0"> 6 12
<STRING, "Acentuação: operação"> 7 13
<STRING, ""> 8 13
```

As aspas fazem parte do lexema, as sequências de escape são mantidas como escritas, e espaços internos e acentos são preservados.

8.6 Testes T07, T08 e T09 — arquivos `.err`

T07:

```
<ERRO_CARACTERE_INVALIDO, @> 4 15
<ERRO_REGISTRADOR_INVALIDO, $t10> 5 4
<ERRO_REGISTRADOR_INVALIDO, $32> 6 6
<ERRO_REGISTRADOR_INVALIDO, $x1> 6 11
<ERRO_REGISTRADOR_INVALIDO, $> 7 10
<ERRO_CARACTERE_INVALIDO, %> 8 9
<ERRO_CARACTERE_INVALIDO, -> 9 15
```

T08:

```
<ERRO_STRING_NAO_FECHADA, "Mensagem nao encerrada> 2 13
<ERRO_ESCAPE_INVALIDO, \q> 3 21
<ERRO_STRING_NAO_FECHADA, "Termina com barra\> 4 13
<ERRO_NUMERO_MALFORMADO, 0xG1> 7 9
<ERRO_NUMERO_MALFORMADO, 0x> 8 9
<ERRO_NUMERO_MALFORMADO, -2abc> 9 9
```

T09:

```
<ERRO_DIRETIVA_INVALIDA, .dados> 1 1
<ERRO_DIRETIVA_INVALIDA, .> 2 1
<ERRO_IDENTIFICADOR_MALFORMADO, soma-total> 4 1
<ERRO_IDENTIFICADOR_MALFORMADO, inicio@> 5 1
<ERRO_IDENTIFICADOR_MALFORMADO, 2valor> 6 1
```

9. Conteúdo final da Tabela de Símbolos

9.1 Teste T01 — símbolos utilizados no programa

A tabela abaixo mostra as entradas cuja posição foi registrada durante a análise do teste T01. As demais palavras reservadas permanecem na tabela com posição `0,0`; o conteúdo integral do arquivo `.ts` está no Anexo A.

Lexema	Categoria	Linha	Coluna
`.data`	diretiva	1	1
`.text`	diretiva	3	1
`.asciiz`	diretiva	2	6
`.globl`	diretiva	4	1
`li`	instrução	6	1
`la`	instrução	9	1
`move`	instrução	12	1
`add`	instrução	8	1
`syscall`	instrução	11	1
`$v0`	registrador	10	4
`$a0`	registrador	9	4
`$t0`	registrador	6	4
`$t1`	registrador	7	4
`$t2`	registrador	8	5
`msg`	identificador/rótulo	2	1
`main`	identificador/rótulo	4	8

Observe que `main` tem primeira ocorrência na linha 4 (`.globl main`), que é uma referência, e não na declaração da linha 5.

9.2 Teste T02 — rótulos e referência antecipada

Lexema	Categoria	Linha	Coluna
`main`	identificador/rótulo	2	1
`teste`	identificador/rótulo	5	3
`laco`	identificador/rótulo	6	1
`fim`	identificador/rótulo	13	5

O rótulo `teste` é referenciado em `j teste` (linha 5) antes de ser declarado (linha 10). A referência antecipada é aceita lexicalmente, e a posição registrada é a da primeira ocorrência.

---

10. Decisões de projeto

Pontos em que o documento é ambíguo ou deixa a escolha ao grupo, e o que foi adotado.

10.1 Linha de comando

O programa recebe 3 argumentos obrigatórios: `./main.exe <entrada.asm> <saida.pre> <saida.lex>`. Os arquivos `.ts` e `.err` são gerados automaticamente com o mesmo nome do `.lex`, trocando apenas a extensão.

10.2 Linha e coluna relativas ao arquivo pré-processado

O analisador léxico lê somente o arquivo `.pre`, como determina a seção 5 do documento. Por isso, linhas e colunas se referem ao `.pre`: como o pré-processador remove indentação, comentários e linhas vazias, essas posições podem diferir das do `.asm` original. Alguns exemplos do documento (ex.: `<INS_LI, li> 3 5`) parecem contar a indentação do código original; optou-se por manter a arquitetura pedida, em que o léxico só conhece o `.pre`.

10.3 Limite de tamanho do lexema

Os lexemas respeitam o limite de 100 caracteres da estrutura `Token` definida no documento (`char lexema[100]`, isto é, 99 caracteres mais o terminador). Caracteres excedentes são descartados.

10.4 Rótulos com nome de palavra reservada

Construções como `add:` e `syscall:` geram `INS_ADD`/`INS_SYSCALL` seguidos de `SMB_COL`, sem erro léxico. Cada lexema é válido isoladamente; a proibição de declarar uma palavra reservada como rótulo é uma regra estrutural, a cargo da análise sintática/semântica (seção 1 do documento).

10.5 Identificadores malformados

Um identificador colado a um caractere que não é delimitador (ex.: `soma-total`, `inicio@`, `valor.x`) gera um único `ERRO_IDENTIFICADOR_MALFORMADO` com a palavra inteira, até o próximo delimitador (espaço, tabulação, quebra de linha, fim de arquivo, `,`, `:`, `(`, `)`, `#` ou `"`). Um lexema iniciado por dígito e seguido de letras (ex.: `2valor`) também gera esse erro, por ser o exemplo de identificador inválido dado pelo documento.

10.6 Números malformados

Hexadecimais com caractere inválido (`0xG1`), prefixo sem dígitos (`0x`) ou decimais negativos seguidos de letras (`-2abc`) geram `ERRO_NUMERO_MALFORMADO`. Prefixo `0X` e dígitos hexadecimais em maiúsculas ou minúsculas são aceitos. Um `-` que não é seguido imediatamente por um dígito gera `ERRO_CARACTERE_INVALIDO`.

10.7 Cadeias de caracteres

- Erros distintos para cadeia não fechada antes da quebra de linha (`ERRO_STRING_NAO_FECHADA`) e antes do fim do arquivo (`ERRO_STRING_NAO_FECHADA_EOF`), pois o documento os lista separadamente.
- O lexema do erro de cadeia não fechada contém todo o conteúdo lido até a quebra de linha.
- Uma cadeia com escape inválido registra `ERRO_ESCAPE_INVALIDO` (com a posição da barra), é lida até a aspa de fechamento e não gera token `STRING`.
- O caso de fim de arquivo não é alcançável pelo `main.exe`, pois o pré-processador termina toda linha com `\n`; foi testado chamando `AnaliseLexica` diretamente.

10.8 Normalização e sensibilidade a maiúsculas

Instruções, diretivas e registradores são reconhecidos sem diferenciar maiúsculas e minúsculas e gravados em minúsculas na Tabela de Símbolos; no `.lex`, o lexema aparece como foi escrito. Rótulos são sensíveis a maiúsculas (`inicio` e `Inicio` são símbolos distintos).

10.9 Primeira ocorrência de rótulos

A Tabela de Símbolos guarda a posição da primeira ocorrência de cada identificador. Em uma referência antecipada (ex.: `j laco` antes de `laco:`), a posição registrada é a da referência.

10.10 Nome do arquivo do analisador léxico

A seção 5.1 do documento mostra o comando `gcc main.c preprocessador.c analisador_lexico.c -o main.exe`, enquanto a tabela de entregáveis e a exigência de declarar `AnaliseLexica` em `lexico.h` usam o nome `lexico.c`. Adotou-se `lexico.c`/`lexico.h`, compilando com `gcc main.c preprocessador.c lexico.c -o main.exe`.

Anexo A — Tabela de Símbolos completa do teste T01 (`T01_correto_soma.ts`)

```
LEXEMA,CATEGORIA,LINHA,COLUNA
.data,diretiva,1,1
.text,diretiva,3,1
.word,diretiva,0,0
.half,diretiva,0,0
.byte,diretiva,0,0
.space,diretiva,0,0
.ascii,diretiva,0,0
.asciiz,diretiva,2,6
.globl,diretiva,4,1
.align,diretiva,0,0
li,instrução,6,1
la,instrução,9,1
move,instrução,12,1
lw,instrução,0,0
sw,instrução,0,0
lb,instrução,0,0
sb,instrução,0,0
lh,instrução,0,0
sh,instrução,0,0
add,instrução,8,1
addi,instrução,0,0
addu,instrução,0,0
sub,instrução,0,0
subu,instrução,0,0
mul,instrução,0,0
div,instrução,0,0
mflo,instrução,0,0
mfhi,instrução,0,0
and,instrução,0,0
or,instrução,0,0
xor,instrução,0,0
nor,instrução,0,0
sll,instrução,0,0
srl,instrução,0,0
slt,instrução,0,0
beq,instrução,0,0
bne,instrução,0,0
blt,instrução,0,0
ble,instrução,0,0
bgt,instrução,0,0
bge,instrução,0,0
```

```
j,instrução,0,0
jal,instrução,0,0
jr,instrução,0,0
syscall,instrução,11,1
nop,instrução,0,0
$zero,registrador,0,0
$v0,registrador,10,4
$v1,registrador,0,0
$a0,registrador,9,4
$a1,registrador,0,0
$a2,registrador,0,0
$a3,registrador,0,0
$t0,registrador,6,4
$t1,registrador,7,4
$t2,registrador,8,5
$t3,registrador,0,0
$t4,registrador,0,0
$t5,registrador,0,0
$t6,registrador,0,0
$t7,registrador,0,0
$t8,registrador,0,0
$t9,registrador,0,0
$s0,registrador,0,0
$s1,registrador,0,0
$s2,registrador,0,0
$s3,registrador,0,0
$s4,registrador,0,0
$s5,registrador,0,0
$s6,registrador,0,0
$s7,registrador,0,0
$k0,registrador,0,0
$k1,registrador,0,0
$gp,registrador,0,0
$sp,registrador,0,0
$fp,registrador,0,0
$ra,registrador,0,0
$0,registrador,0,0
$1,registrador,0,0
$2,registrador,0,0
$3,registrador,0,0
$4,registrador,0,0
$5,registrador,0,0
$6,registrador,0,0
$7,registrador,0,0
$8,registrador,0,0
$9,registrador,0,0
$10,registrador,0,0
$11,registrador,0,0
$12,registrador,0,0
$13,registrador,0,0
$14,registrador,0,0
$15,registrador,0,0
$16,registrador,0,0
$17,registrador,0,0
$18,registrador,0,0
$19,registrador,0,0
$20,registrador,0,0
$21,registrador,0,0
$22,registrador,0,0
$23,registrador,0,0
$24,registrador,0,0
$25,registrador,0,0
$26,registrador,0,0
$27,registrador,0,0
$28,registrador,0,0
$29,registrador,0,0
$30,registrador,0,0
$31,registrador,0,0
msg,identificador/rótulo,2,1
main,identificador/rótulo,4,8
```